

逻辑回归与大学排名分类

本实验用学校的部分信息，判断它是否进入 QS 2026 榜单前 500 名。我们将完成数据处理、模型训练和分类评价，理解一个分类器怎样从有答案的样本中学习。

课堂安排：理论回顾约 15 分钟，项目操作约 10 分钟，随后独立完成实验。

一 逻辑回归理论回顾

1 分类问题中的输入和答案

每行数据代表一所学校。模型的输入叫**特征**，已知答案叫**标签**，本实验标签为：

$$y = \begin{cases} 1, & \text{学校进入前 500 名} \\ 0, & \text{学校在榜单中，但未进入前 500 名} \end{cases}$$

标签保存在 `top500` 列。输入由八个字段组成：

类型	学校信息
3 个数值字段	师生比指标得分、国际教师指标得分、国际学生指标得分
5 个类别字段	地区、办学性质、学校规模、学科覆盖类型、研究活跃程度

训练时，模型同时看到特征和标签，学习二者的关系；预测时，它根据特征给出属于正类的概率。这是**有监督二分类**。这里使用同年信息识别同年排名类别，结果对应这份榜单中的学校。

2 从线性得分到分类概率

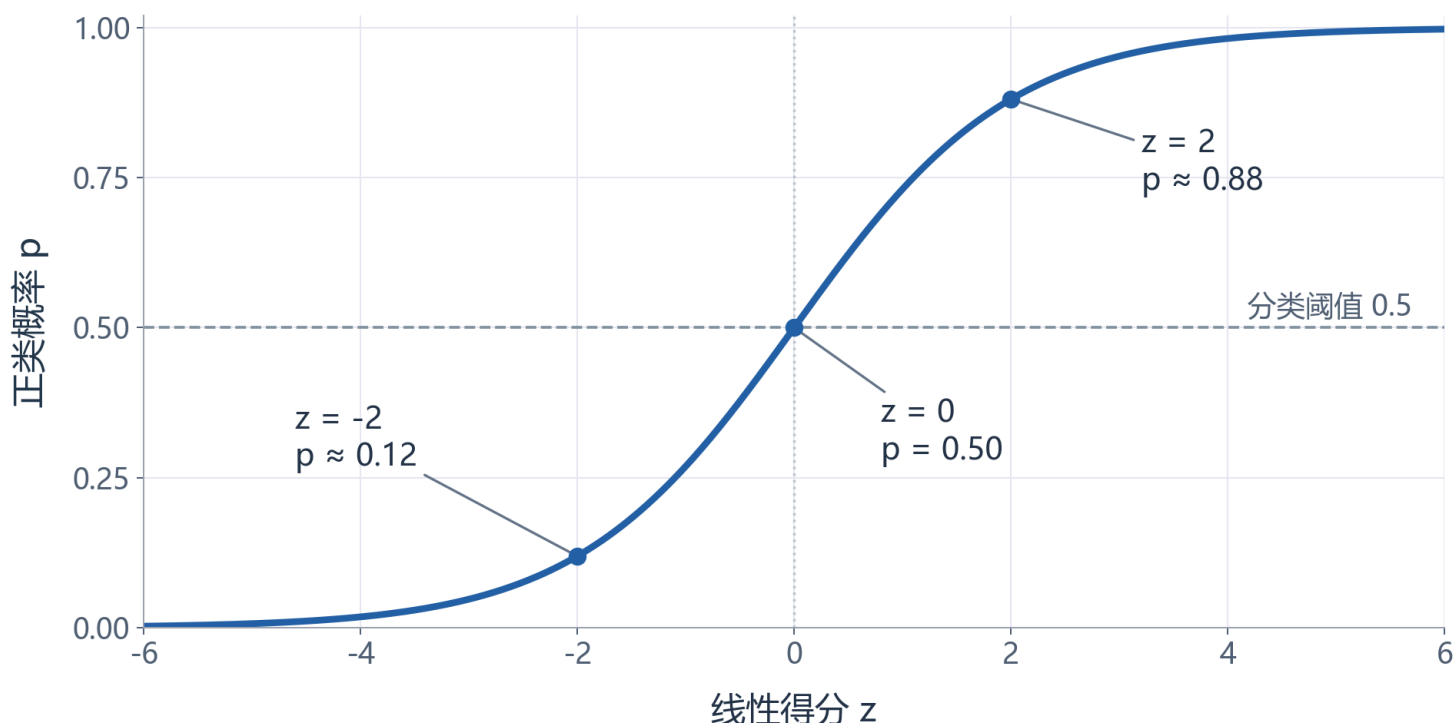
先为一所学校计算加权得分：

$$z = b + \boldsymbol{w}^\top \boldsymbol{x} = b + \sum_{j=1}^d w_j x_j$$

$\boldsymbol{x}$ 是处理后的 d 个特征， w_j 是对应权重， b 是截距，表示共同的基础倾向。**权重和截距是训练要学到的参数**。得分 z 可以取任意实数，再由 sigmoid 函数转成概率：

$$p = P(y = 1 \mid \boldsymbol{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

sigmoid: 得分越大, 正类概率越高



得分为 $-2, 0, 2$ 时, 概率约为 $0.12, 0.50, 0.88$ 。曲线越往右, 模型越倾向于将学校判为前 500 名。

设置阈值 t 后, 预测类别 $\hat{y}$ 为:

$$\hat{y} = \begin{cases} 1, & p \geq t \\ 0, & p < t \end{cases} \quad \text{本实验主实验取 } t = 0.5$$

例如, 一所假想学校的两个特征是 $(1, 2)$, 权重为 $(0.8, 0.6)$, 截距为 -1 , 则 $z = -1 + 0.8 \times 1 + 0.6 \times 2 = 1$, 概率约为 0.73 , 预测类别为 1 。**得分、概率、类别是三个不同的输出。**

固定其他特征时, 一个正权重会使对应特征越大的学校得分越高, 负权重则相反。这表达的是模型学到的关联; 例如国际学生指标得分的权重为正, 并不意味着增加国际学生一定能提高排名。

项目把每行特征前面加一个常数 1 , 将参数合成 $\theta = (b, w_1, \dots, w_d)^\top$ 。于是, 一批样本可统一写为 $z = X\theta$ 。编码后有 28 个特征, 加上常数列共 29 列: 一批 64 所学校对应 X 的形状为 $(64, 29)$, 参数形状为 $(29, 1)$, 输出 64 个得分。

3 从真实标签的概率到交叉熵

模型给出的 p 越接近真实标签, 通常预测越好。怎样把“好坏”变成可以优化的数值? 对于一个样本, 真实标签出现的概率可以统一写成:

$$P(y | \mathbf{x}) = p^y (1 - p)^{1-y}$$

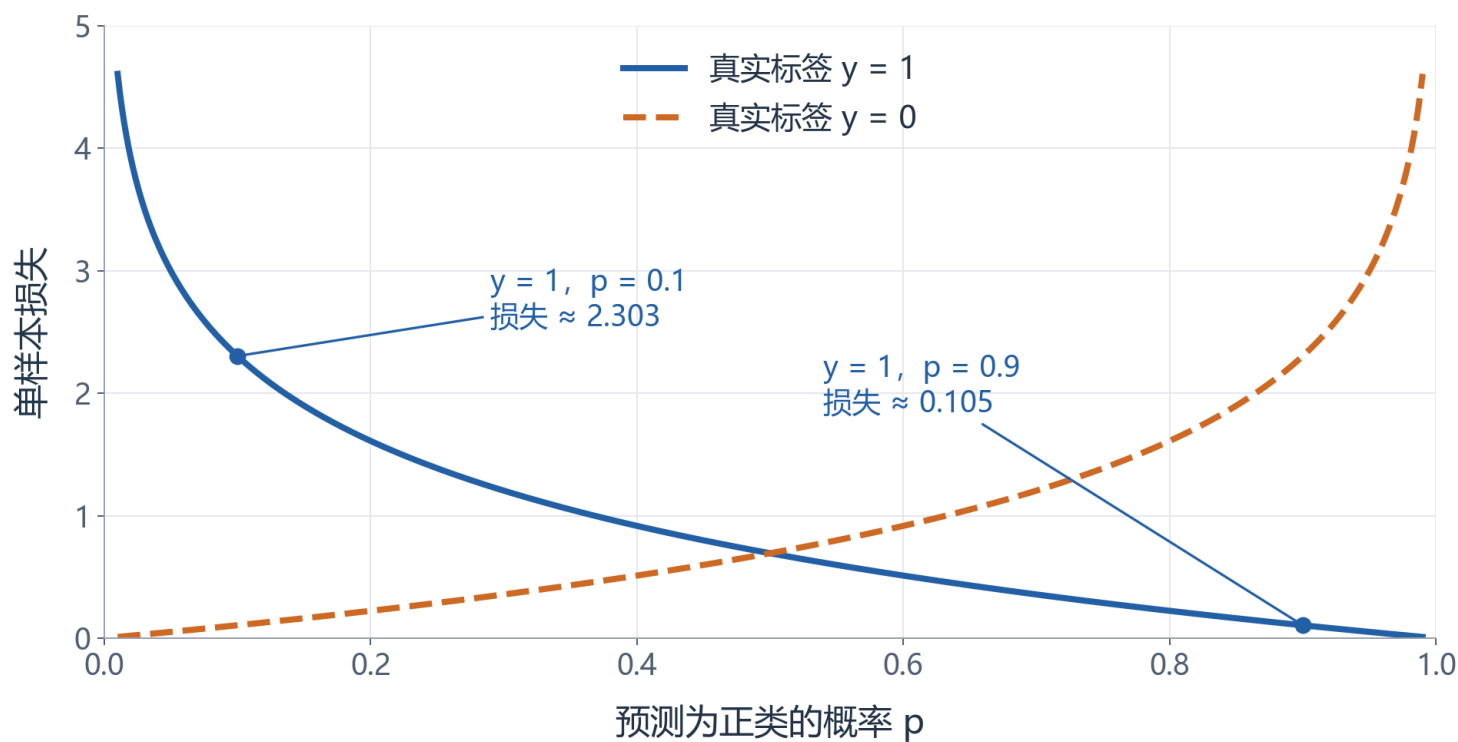
当 $y = 1$ 时，这个式子等于 p ；当 $y = 0$ 时，等于 $1 - p$ 。在样本独立的假设下，一批真实标签的概率是这些项的乘积，这就是**似然**。最大化似然，等价于最小化它的负对数；对数又能把乘积转成求和。

因此，单个样本的二元交叉熵和一批 B 个样本的平均损失为：

$$\ell(p, y) = -[y \log p + (1 - y) \log(1 - p)]$$

$$L_B = \frac{1}{B} \sum_{i=1}^B \ell(p_i, y_i)$$

交叉熵：自信地预测错误，损失更大



真实标签为 1 时，给出 $p = 0.9$ 的损失约为 0.105，给出 $p = 0.1$ 的损失约为 2.303。**越自信地预测错误，损失越大**。标签为 0 时则相反，希望 p 接近 0。

训练代码中的 `BCEWithLogitsLoss` 接收原始得分，内部合并计算 sigmoid 和交叉熵。因此模型的 `forward` 返回得分；计算分类指标时，再把得分转换为概率。

4 梯度怎样改变参数

训练要调整的是权重和截距，因此最终需要计算**损失对参数的导数**。先看一个样本、不加正则化的情况。前向计算依次经过：

$$\underbrace{w_1, \dots, w_d, b}_{\text{参数}} \longrightarrow \underbrace{z = b + \sum_j w_j x_j}_{\text{得分}} \longrightarrow \underbrace{p = \sigma(z)}_{\text{概率}} \longrightarrow \underbrace{\ell(p, y)}_{\text{损失}}$$

权重 w_j 先影响得分，得分影响概率，概率再影响损失。所以要用链式法则，把沿途的导数乘起来：

$$\boxed{\frac{\partial \ell}{\partial w_j} = \frac{\partial \ell}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}}$$

第一步：损失对概率求导。 二元交叉熵为 $\ell = -[y \log p + (1 - y) \log(1 - p)]$ ，所以：

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

第二步：概率对得分求导。 因为 $p = \sigma(z)$ ，sigmoid 的导数为：

$$\frac{\partial p}{\partial z} = p(1 - p)$$

把前两步的导数相乘，就得到损失对得分的导数：

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial p} \frac{\partial p}{\partial z} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p) = p - y$$

这里的 $p - y$ 只是中间结果。我们还要继续求到参数，才能更新权重。

第三步：得分对参数求导。 得分是 $z = b + \sum_j w_j x_j$ 。对某个权重 w_j 求导时，输入 x_j 是已知常数，因此：

$$\frac{\partial z}{\partial w_j} = x_j, \quad \frac{\partial z}{\partial b} = 1$$

继续应用链式法则，最终得到：

$$\boxed{\frac{\partial \ell}{\partial w_j} = (p - y)x_j}, \quad \boxed{\frac{\partial \ell}{\partial b} = p - y}$$

也就是说，每个权重的梯度都要把 $p - y$ 再乘上对应特征。截距的导数乘的是 1，所以数值恰好仍为 $p - y$ 。

第四步：用参数梯度更新参数。 沿梯度反方向走一小步：

$$\begin{aligned} w_j &\leftarrow w_j - \eta (p - y)x_j \\ b &\leftarrow b - \eta (p - y) \end{aligned}$$

η 是**学习率**，控制每步调整幅度。右侧的梯度都由更新前的同一组参数计算。学习率太小时变化慢，太大时损失可能波动甚至增大。

例如只有一个特征 $x = 2$ ，真实标签 $y = 1$ ，初始 $w = b = 0$ ，因此 $z = 0$ 、 $p = 0.5$ 。损失对得分的导数是 -0.5 ；对权重的导数则是 $-0.5 \times 2 = -1$ 。取学习率 0.1 ，更新后 $w = 0.1$ 、 $b = 0.05$ ，新得分为 0.25 ，概率约为 0.562 ，朝真实标签 1 靠近了。

一批样本怎样计算？先分别求导，再取平均。 对一批 B 个样本，平均损失为 $L_B = \frac{1}{B} \sum_i \ell_i$ ，所以某个权重的梯度为：

$$\frac{\partial L_B}{\partial w_j} = \frac{1}{B} \sum_{i=1}^B (p_i - y_i) x_{ij}$$

x_{ij} 表示第 i 个样本的第 j 个特征。这个式子就是把刚才每个样本的 $(p - y)x_j$ 加起来，再除以样本数。

将所有参数的梯度一起写成向量，就得到代码对应的矩阵形式：

$$\mathbf{g}_B = \nabla_{\boldsymbol{\theta}} L_B = \frac{1}{B} \mathbf{X}_B^\top (\mathbf{p}_B - \mathbf{y}_B), \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{g}_B$$

$\mathbf{X}_B$ 的每行是一条样本，第一列全为 1 ，因此矩阵乘法同时算出了截距和各个权重的梯度。它把逐个样本、逐个参数的计算合并成了一次运算。

每次用一批样本更新一次参数，称为小批量训练；完整遍历一次训练集，称为一轮，即一个 epoch。一批样本共享参数，梯度综合了多个样本的要求，因此一次更新可能改善部分样本、同时使另一些样本的预测变差。

C6 手算例子：两条样本的一次更新。 两所假想学校的数据为：

$$\mathbf{X}_B = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad \mathbf{y}_B = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

初始得分均为 0 ，概率均为 0.5 ，所以两个样本的 $p - y$ 分别是 -0.5 和 0.5 。分别乘上各自的增广特征，再取平均：

$$\mathbf{g}_B = \frac{1}{2} \left[-0.5 \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} + 0.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \right] = \begin{pmatrix} 0 \\ -0.25 \\ 0.25 \end{pmatrix}$$

取学习率 0.4 ，一步更新后：

$$\boldsymbol{\theta}_{\text{new}} = (0, 0.1, -0.1)^\top$$

两所学校的新概率约为 0.525 和 0.475 ，都朝正确标签移动。C6 要手算并核对这个过程。实际训练时，PyTorch 的反向传播沿计算图应用链式法则，得到损失对参数的梯度；优化器再根据这些梯度更新

参数。

5 L1 与 L2 正则化

为了限制权重过大，可以在平均数据损失上增加一项惩罚：

$$J_B = L_B + \lambda R(\mathbf{w}), \quad R_{L1}(\mathbf{w}) = \sum_j |w_j|, \quad R_{L2}(\mathbf{w}) = \frac{1}{2} \sum_j w_j^2$$

λ 控制惩罚强度，训练时对总目标 J_B 求梯度。L2 倾向于缩小权重；L1 倾向于产生稀疏权重，但本项目采用的普通 SGD 不保证权重恰好变为 0。截距表示整体基础倾向，本实验只惩罚特征权重。

项目比较无正则、L2 ($\lambda = 0.01$)、L1 ($\lambda = 0.01$) 三个方案。正则化能否改善效果，需要由验证集表现判断。

惩罚过强也可能限制模型学习有用的信息。比较方案时，各组采用相同的数据划分和训练设置，才能把观察到的差别与正则化设置联系起来。

6 怎样评价分类结果

将预测和真实标签交叉统计，得到混淆矩阵。这里正类是前 500 所学校：

真实类别	预测为 0	预测为 1
真实为 0	TN：正确判为负类	FP：误报
真实为 1	FN：漏判	TP：正确找到正类

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}$$

准确率表示整体猜对的比例；精确率问“选出来的有多少是真的”，召回率问“真的里面找到了多少”，F1 综合衡量精确率与召回率。

例如实际有 20 所正类学校，模型选出 10 所，其中 8 所正确，则精确率为 $8/10 = 80\%$ ，召回率为 $8/20 = 40\%$ ，F1 约为 0.533。模型选得较准，但遗漏较多。本实验约三分之二的学校为负类，所以还需和“全部预测为负类”的多数类基线比较。

降低阈值会选中更多学校，召回率不会下降，也可能增加误报。实验将比较 0.3、0.5、0.7 三个阈值；指标分母为 0 时，代码约定相应比率为 0。

二 项目操作指南

1 数据准备与处理顺序

课堂数据已放在 `code/data/classroom/`，按学校划分为互不重叠的三份：

数据集	学校数	用途
训练集 train	902	学习预处理统计量和模型参数
验证集 val	300	比较训练轮次和模型方案
测试集 test	302	方案确定后评价最终效果

按以下顺序处理每份数据：

1. 过滤标签缺失的行，把数值列中的空白、横线等转换为缺失值。
2. 用训练集各列均值填补数值缺失，再减均值、除以标准差，完成标准化。
3. 对类别字段进行独热编码，缺失和未见类别放入 `Unknown`。
4. 拼接数值与类别特征，转换成张量，在特征首列添加常数 1。

例如类别表为“亚洲、欧洲、Unknown”，欧洲编码为 $(0, 1, 0)$ 。**均值、标准差和类别表只从训练集得到，验证和测试沿用它们。** 数值列标准化，独热列保留 0/1。

标准化可以理解为“离训练均值有几个标准差”。例如训练列为 40、60，均值为 50，总体标准差为 10，标准化后为 -1 、 1 ；验证集中出现 80，仍使用这组统计量，得到 3。采用统一尺度，模型在训练与预测时才能一致地解释输入。

独热编码也使用训练时确定的列顺序。若训练中的第二列表示欧洲，验证数据的第二列也必须表示欧洲，这样同一个权重才始终对应同一种信息。

标签是学习的答案；学校名称和原表行号仅用于识别记录。项目已选定八个输入字段，实际排名和总分也不作为输入。真实数据的标签均有效，缺失标签的处理在 C1 小例子中检查。

2 C0 到 C7 的任务

主要补全 `student.py` 中的 17 处任务。每个函数的注释给出了输入、输出与提示；Notebook 按阶段调用这些函数。读数据、画图和比较方案的辅助程序已经提供。

阶段	待完成函数	工作内容与自检结果
C0	已给例子，无需补全	观察原表 (902,11) 和矩阵乘法；11 列包含名称、行号、标签
C1	filter_labels clean_numeric	过滤缺失标签，清洗数值；无效文本变缺失，数值 0 保留
C2	fit_statistics apply_statistics fit_categories encode_categories to_tensors	完成填补、标准化、独热编码与张量转换； 完整训练输入形状为 (902,29)
C3	UniversityDataset.__len__ UniversityDataset.__getitem__ make_loader	按相同索引取特征和标签，再分批；5 条样本每批 2 条，应得到 [2,2,1]
C4	sigmoid LogisticRegression.__init__ LogisticRegression.forward	注册参数、计算得分与概率；模型返回线性得分
C5	data_loss LogisticRegression.penalty	实现交叉熵和正则惩罚；零得分的平均损失约为 0.6931
C6	train_epoch	完成清梯度、前向、反向、更新及损失累计； 与一次手算更新核对
C7	binary_metrics	返回四种计数和四项指标，再运行完整训练流程

按顺序推进即可。C5 需要 C4 的模型，C6 需要前面建立的数据批次、模型和损失函数；完整训练需要所有阶段完成。

3 运行与自检

以 Notebook 为主要入口：

- 1. 打开 code/logic.ipynb ，选择已安装 pandas、PyTorch 等依赖的课程 Python 内核。
- 2. 先运行初始化单元格，确认输出的 student.py 路径就是正在编辑的文件。
- 3. 修改对应函数，按 **Ctrl+S 保存**，再运行该阶段的 Notebook 单元格。
- 4. 看到 C1 PASS 等输出后继续。它表示该阶段的小例子通过，报告还需记录具体结果与解释。

初始化已启用自动加载，保存后的代码会在后续运行时生效。修改前面阶段后，从该阶段顺序重跑，以重新生成后续使用的数据和模型。

也可以在已激活课程环境的终端进入 `code/`，运行：

```
python checkpoints.py C1
python checkpoints.py C2
python checkpoints.py all
```

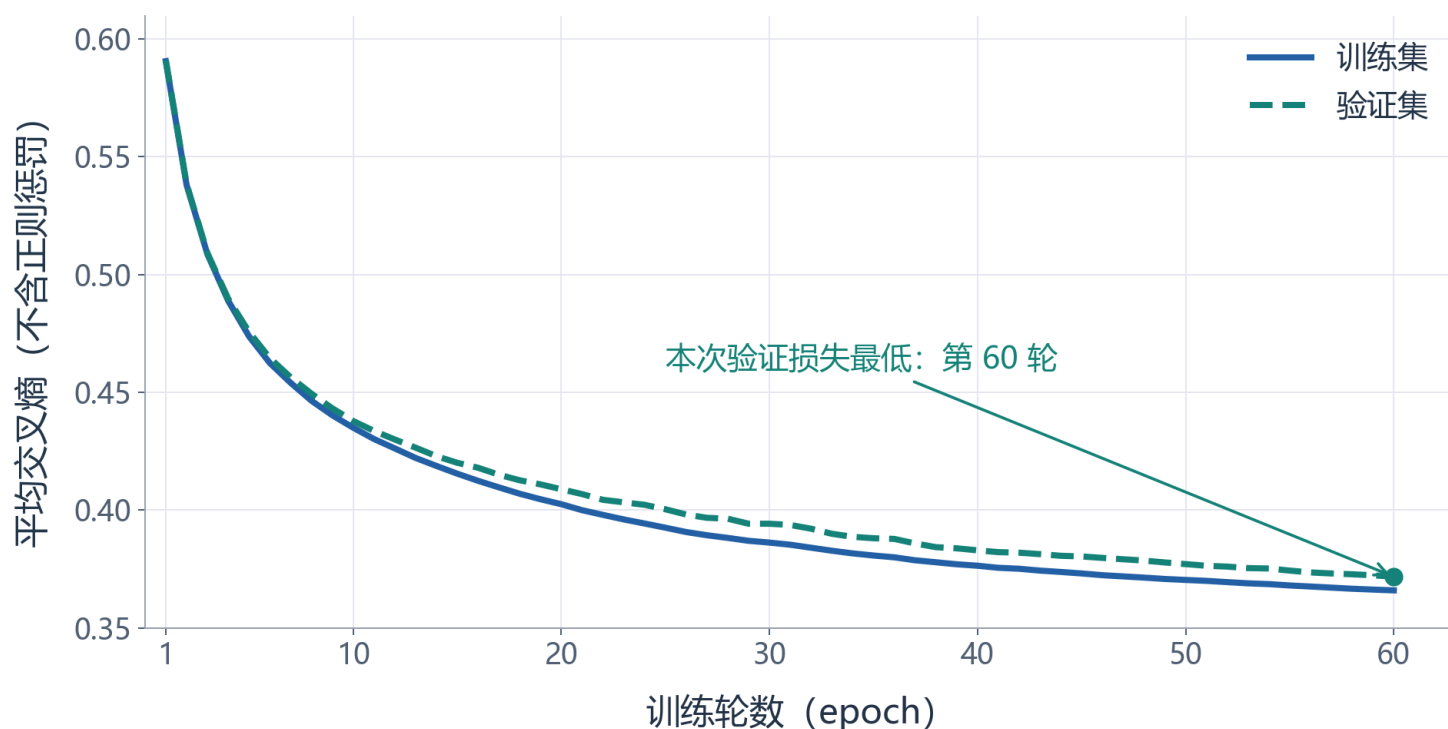
将阶段编号换成当前要检查的阶段即可。出现“先完成 C...”时，查看对应函数是否仍返回 `None`，并确认文件已保存；变量未定义时，先运行初始化与前序单元格；找不到 `pandas` 或 `torch` 时，检查选用的 Python 环境。

4 完成训练并阅读结果

所有阶段完成后，运行 C7 的完整实验单元格。程序为三种方案分别训练 60 轮，每批 64 条、学习率 0.05、随机种子 42；训练集一轮共 15 批，最后一批 6 条。前面的手算例子使用学习率 0.4，是为了方便核对。

程序先保留每个方案**验证数据损失最低的一轮**，再从三个方案中选择验证损失最低的模型。主实验分类阈值固定为 0.5。

无正则方案：训练与验证损失



图中是本项目无正则方案的一次试运行：横轴是轮数，纵轴是平均交叉熵，越低越好。两条曲线均在每轮结束后计算，不含正则惩罚；本次验证损失在第 60 轮最低。

如果训练损失继续下降，而验证损失持续上升，说明对训练数据的适应没有带来更好的验证表现，是过拟合的一种信号。准确率相同的模型，也可能给出不同的预测概率，因此交叉熵仍可能不同。

Notebook 默认将结果保存到 `code/outputs-qs-notebook/`。查看方案比较表 `candidates.csv`、所选方案 `selection.json` 和损失图；再用 `validation_thresholds.csv` 比较阈值变化对精确率、召回率的影响。

记录所选模型和阈值后，将最终测试单元格的 `RUN_FINAL_TEST` 改为 `True` 并运行。此后用测试结果报告效果，模型选择始终依据验证集。

若使用终端，完整训练和最终测试分别运行以下命令，结果默认保存到 `code/outputs-qs/`：

```
python run_lab.py
# 查看验证结果并记录所选方案后，再运行：
python run_lab.py --final-test
```

5 实验报告与提交

报告按“实验目的、环境与数据、原理、内容与步骤、结果、分析与讨论、总结”组织，重点保留以下证据：

- **阶段任务**：说明 C0–C7 的完成情况，给出关键代码、小例子的输入与结果；C2 包括数值处理和类别编码。
- **一次手算**：给出 C6 的输入、初始概率、梯度和更新后的参数，并与程序结果核对。
- **实验结果**：三种方案的验证比较表、损失曲线、最终测试的混淆矩阵与准确率、精确率、召回率、F1，以及多数类基线。
- **结果分析**：解释正则化是否改善表现，结合至少两个验证阈值讨论误报与漏判，并说明本任务使用同年数据等局限。

图表使用本人运行的结果，配上解释。关键代码摘录通常每处 3–10 行即可，图表编号后在正文引用。最终测试结果不能用来反复调整模型和阈值。

选做：去掉 `focus`、`research` 两个字段，固定 L2 系数 0.01，在验证集比较完整与精简特征。Notebook 的 `RUN_FEATURE_STUDY` 开关控制此实验，安排在最终测试之前。

提交 `学号_姓名_lab03.zip`，包含 `report.pdf` 和可运行的 `code/`。代码目录保留实现、运行入口、辅助模块、依赖说明和三份课堂数据；使用 Notebook 时一并提交 `logic.ipynb`。无需打包 Python 环境和缓存。报告首页注明姓名、学号、实验名称；截止时间与提交渠道以课程通知为准。